

Labwork 3

Le Quoc Trung
Student ID: 2540103

September 30, 2026

1 Goal

Convert an RGB image to grayscale on the CPU and on the GPU with Numba CUDA, then compare the running time. The grayscale value of a pixel is the average of its three channels. The labwork was done on Google Colab with a Tesla T4.

2 Implementation

The image is loaded with `imread`, then reshaped into a 1D array of shape `(pixelCount, 3)`. Working on a flat array means one thread can find its pixel with a single index instead of a pair of coordinates.

The CPU version is a simple loop over all pixels. For each one it sums the three channels, divides by three, and writes the same value back to the three output channels. The values are cast to integers before the sum, otherwise the three 8-bit channels overflow.

The GPU version has no loop. The kernel is annotated with `@cuda.jit` and every thread computes its own position from its thread index, its block index and the block size. The number of blocks is the number of pixels divided by the block size, rounded up. Since 271523 is not a multiple of the block size, the last block has a few threads with no pixel to process, so the kernel first checks that its index is still inside the array before writing anything.

On the host side the program follows the four steps of a CUDA program: the input array is copied to the device, an output buffer is allocated there, the kernel is launched with the chosen grid and block size, and the result is copied back to the host. Both versions give exactly the same output image, which I checked by comparing the two arrays.

3 Results

With a block size of 64, the CPU takes 0.5287 s and the GPU 0.000298 s, which is a speedup of about 1775 times.

I then ran the kernel with block sizes from 1 to 1024. Each value is the average of 20 launches, after one warm-up launch, because the first call also includes the time Numba needs to compile the kernel.

Block size	Time (ms)
1	2.119
2	1.063
4	0.536
8	0.273
16	0.141
32	0.075
64	0.075
128	0.075
256	0.075
512	0.079
1024	0.083

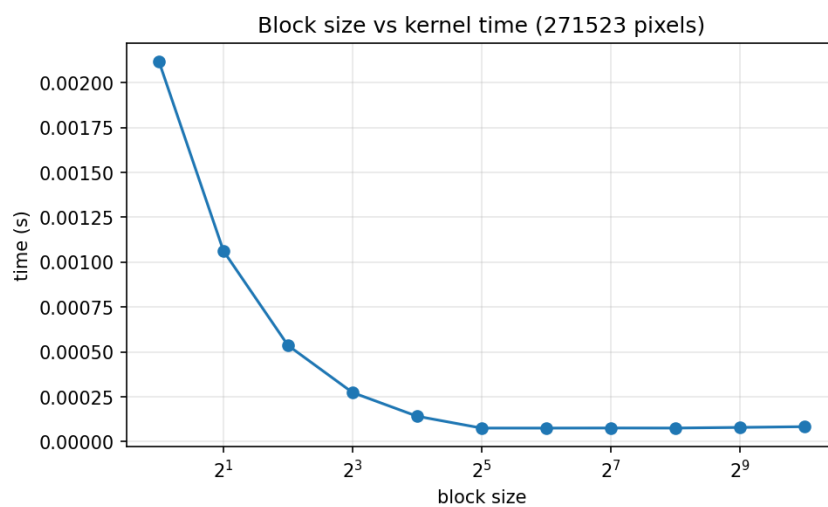


Figure 1: Block size vs kernel time